PowerPoint about XSS made by Kasra Zahedi

What is XSS?

Cross-Site Scripting (XSS) is a type of cyberattack where an attacker injects malicious code, usually JavaScript, into a website that other people trust.
XSS happens when a website accepts user input and displays it without properly checking or encoding it. The victim's browser may then execute the malicious code as if it were legitimate.
For example, an attacker could put malicious code into a website's comment section. When another user views the comment, the code might execute in their browser.

What damage does XSS do?

An attacker may be able to:

- Steal sensitive information, including accessible session cookies.
- Hijack a user's session and potentially take over their account.
- Redirect users to malicious websites.
- Modify website content to display false information.
- Access information or perform actions available to the victim.
- Target administrators or other users with elevated privileges.

The impact can range from minor website manipulation to serious account compromise.

Examples of XSS:

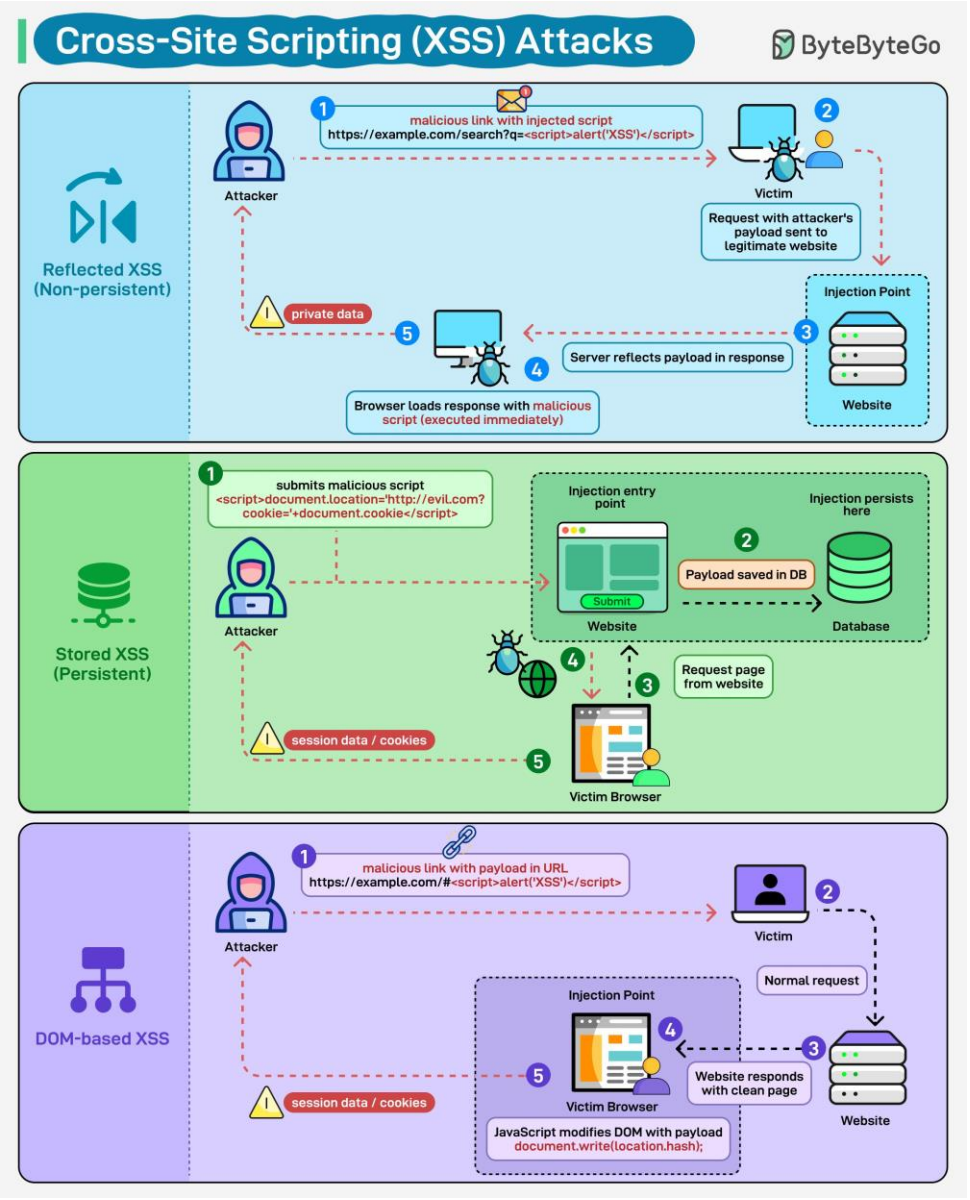


Image from https://blog.bytebytego.com/p/ep184-api-vs-sdk

Reflected XSS

Malicious input is sent to a website, often through a specially crafted link, and immediately included in the response without proper protection.

Example: A victim clicks a malicious link, and the website returns a page containing code that executes in their browser.

Stored XSS

Malicious code is saved on the server, such as in a database, forum post, comment, or user profile. It can execute whenever another user views the affected content.

Example: An attacker submits a malicious comment that affects everyone who opens the page.

DOM-based XSS

Malicious input is processed by unsafe client-side JavaScript, which modifies the webpage in a way that allows code to execute.

Example: JavaScript takes untrusted data from a URL and inserts it into the page unsafely.

How to prevent XSS?

Some main examples of prevention methods are:

- **Output encoding**: Convert special characters into safe representations appropriate to where the data appears, so they are treated as text rather than executable code.
- **Input validation**: Check that submitted data follows the expected format.
- **Safe JavaScript practices**: Avoid unsafe methods that insert untrusted input directly into HTML.
- **Security reviews and testing**: Check code for places where user input reaches webpage output, and test for vulnerabilities.
- **Content Security Policy (CSP):** Use a restrictive policy as an additional layer of protection.
- **Secure session handling**: Protect cookies with appropriate security settings to reduce the risk of session theft.



Image from www.magnific.com